

- **Intent:** **behavioral** design pattern that lets you define a subscription mechanism to notify multiple objects about any events that happen to the object they're observing
- **Problem:**
 - Have two types of objects: Customer and Store
 - Customer is interested in specific brand of product
 - Can check at store everyday for availability→ but while product is being shipped, the trips are useless
 - Store could send emails for when it is available→ not necessary for uninterested customers
 - Either customer wastes time checking availability or store wastes resources notifying all customers
- **Solution:**
 - Object that has some interesting state is often called subject→ but since it will also notify other objects about changes to its state, we will call it publisher→other objects that want to track changes to publisher's state are called subscribers
 - Pattern suggests that you add subscription mechanism to publisher class so individual objects can subscribe to or unsubscribe from a stream of events coming from that publisher
 - Mechanism consists of
 - Array field for storing a list of references to subscriber objects
 - Several public methods which allow adding subscribers to and removing them from that list
 - When important event happens to publisher, it goes to subscribers and calls specific notification method on objects
- **Structure:**

UML Relations

Association: These classes know each other.

Aggregation: A whole uses parts, but the parts can exist independently.

Composition: A whole owns parts, and the parts cannot exist without the whole.

CREATIONAL PATTERNS

Singleton — Only one instance, global access

Idea: Ensure a class has *only one* instance.

Real-world example:

There's *only one* active database connection manager in an app.

Abstract Factory — Creates families of related objects

Idea: Create sets of related objects without specifying concrete classes.

Real-world example:

UI theme: LightThemeFactory → Light buttons, Light menus; DarkThemeFactory → Dark buttons, Dark menus

Factory Method — Subclasses decide what object to create

Idea: Base class defines a method for creating objects, but subclasses choose the type.

Real-world example:

A DocumentEditor creates documents: TextEditor → TextDocument; ImageEditor → PhotoDocument

Builder — Construct complex objects step-by-step

Idea: Useful for objects with *many* configuration options.

Real-world example: Building a pizza: choose crust, choose sauce, choose toppings

STRUCTURAL PATTERNS

Composite — Tree structure; treat part and whole uniformly

Idea: Represent hierarchies where objects contain other objects.

Real-world example:

A file system: Folder contains files or other folders, both support operations like size()

Bridge — Decouple abstraction from implementation

Idea: Split "what it is" from "how it works" so they can vary independently.

Real-world example:

A RemoteControl works for LG TV and Sony TV

Same remote operations, different device implementations.

Proxy — Placeholder controlling access to another object

Idea: A "middleman" object that controls access to a real object.

Real-world example:

Credit card (proxy) → Bank account (real subject), Virtual proxy loads a large image only when needed

Decorator — Add responsibilities dynamically

Idea: Wrap an object to add behavior at runtime instead of subclassing.

Real-world example:

Start with a plain coffee then add Milk Decorator, Caramel Decorator, WhippedCream Decorator

BEHAVIORAL PATTERNS

Command — Encapsulate a request as an object

Idea: Wrap actions in objects so they can be: queued, logged or undone

Real-world example: Undo/Redo in a text editor.

Observer — Notify dependents on state change

Idea: One object updates many observers automatically.

Real-world example: You subscribe to a YouTube channel, When they upload a video → You get notified

- **Diagram:**
 - 1. Publisher: subscribers: Subscriber[]; -mainState; +subscriber(): Subscriber; +unsubscribe(): Subscriber; +notify(): Subscriber[]; +mainBusinessLogic();
 - 2. Renewal (p is subscriber): +unsubscribe(); +unsubscribe(): Subscriber[]; +mainState = newState; notify(subscribers);
 - 3. ConcreteSubscribers: +update(): Content;
 - 4. Client: s = new ConcreteSubscribers(); publisher.subscribe(s);
 - 5. Client: +update(): Content;
 - 6. Client: +update(): Content;
 - **Publisher:** issues events of interest to other objects→events occur when publisher changes its state or executes some behaviors→ publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list
 - When new event happens publisher goes over the subscription list and calls the notification method declared in subscriber interface
 - **Subscriber interface:** declares notification interface→mainly consists of update() that may have several parameters that let the publisher pass some event details with update
 - **Concrete subscribers:** perform some actions in response to notification issued by publisher→all classes must implement same interface so publisher isn't coupled to concrete classes
 - **Client:** creates publisher and subscriber objects separately and then registers subscribers for publisher updates
- **Analogy:**
 - Newsletter subscription service
 - Subject:newsletter publisher
 - Observers: subscribers who notifications
 - When publisher releases a new issue, it notifies all subscribers so they receive the content
- **BASIC: SUBJECT MAINTAINS LIST OF OBSERVERS AND PROVIDES METHODS TO ADD, REMOVE, AND NOTIFY OBSERVERS—OBSERVER INTERFACE DEFINES INTERFACE WITH UPDATE() WHICH IS CALLED WHEN SUBJECT STATE CHANGES**

State:

• **Intent:** **behavioral** design pattern that lets an object alter its behavior when its internal state changes→appears as if the object changed its class

- **Problem:**
 - At any given moment, there's a finite number of states which a program can be in
 - Within any unique state, the program behaves differently; and the program can be switched from one state to another
 - Weakness of a state machine based on conditionals reveals itself once we add more and more states and state-dependent behaviors
 - Difficult to predict all possible states and transitions at the design stage
- **Solution:**
 - Pattern suggests you create new classes for all possible states of an object and extract all state-specific behaviors into these classes
 - Original object called context, stores a reference to one of the state objects that represents its current state, and delegates all state-related work to that object
- **Structure:**

SPACE COUNT FROM FILE	FIND LONGEST COMMON PREFIX	VALID PARENTHESIS CHECKER	REMOVE MATCHING STRING POINTERS
<pre>#include <iostream> #include <string> using namespace std; int CountWords(char const *fileIn) { ifstream f(fileIn); if (!f.is_open()) return 0; string word; while (f.getline(word, 100)) { int count = 0; for (int i = 0; i < word.length(); i++) { if (word[i] == ' ') count++; } count++; } return count; }</pre>	<pre>string &LongestPrefix(const vector<string> &words) { if (words.empty()) return ""; string prefix = words[0]; for (int i = 1; i < words.size(); i++) { while (prefix.substr(0, prefix.length()) != words[i].substr(0, prefix.length())) { prefix = prefix.substr(0, prefix.length() - 1); } } return prefix; }</pre>	<pre>bool &ValidParenthesisCheck(const string &str) { string par; for (char c : str) { if (c == '(' c == '[' c == '{') { par.push_back(c); } else if (c == ')' c == ']' c == '}') { if (par.empty() par.back() != c) return false; par.pop_back(); } } return par.empty(); }</pre>	<pre>void removeMatchingStringPointers(vector<string> &strs, vector<int> &indices) { for (int i = 0; i < str.size(); i++) { if (str[i] == "(" str[i] == "[" str[i] == "{") { indices.push_back(i); } } for (int i = 0; i < indices.size(); i++) { str.erase(str.begin() + indices[i]); } }</pre>

In object oriented design, **polymorphism** allows developers to assemble objects with well-designed interfaces without needing to know the internal details of those objects.

The **state** design pattern is typically used when you want to allow an object to alter its behavior when its internal state changes.

The command pattern encapsulates a request as an **object**, allowing for parameterization, queuing, and undoability.

<pre>public Lightbulb(Lightbulb &lb) : bulb(&lb) {} virtual void Push() override { bulb.SetOn(); bulb.IsOn(); } private: Lightbulb &bulb; }; class PushButton : public Button { public: PushButton(Lightbulb &lb) : lb(&lb) {} void Push() override { lb.Push(); } };</pre>	<pre>{ bulb = &b; } void Push() { bulb.SetOn(); bulb.IsOn(); } private: Lightbulb &bulb; };</pre>
--	---